A PROJECT REPORT ON

INDOOR AIR QUALITY AND ENVIRONMENTAL DATA LOGGER WITH DATA ANALYSIS AND MACHINE LEARNING

Submitted By

Name: Bishnujyoti Gogoi

Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the project report entitled “Indoor Air Quality and Environmental Data Logger with Data Analysis and Machine Learning” is a bona fide record of the work carried out by Bishnujyoti Gogoi, a student of the Department of Electronics and Communication Engineering, Jorhat Institute of Science and Technology

The work presented in this report has been carried out under my supervision and, to the best of my knowledge, has not been submitted elsewhere for the award of any other degree or diploma.

Date: [DATE]

Place: [PLACE]

_____

[PROJECT GUIDE NAME]

[DESIGNATION], Department of Electronics and Communication Engineering

_____

[HEAD OF DEPARTMENT NAME]

Head of Department, Electronics and Communication Engineering

DECLARATION

I hereby declare that the project report entitled "Indoor Air Quality and Environmental Data Logger with Data Analysis and Machine Learning" submitted by me to the Department of Electronics and Communication Engineering, Jorhat Institute of Science and Technology.

I further declare that this work has not been submitted, either in part or in full, to any other university or institute for the award of any degree or diploma, and that all sources of information used in the preparation of this report have been duly acknowledged.

Date:

Place:

_____

Bishnujyoti Gogoi

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide Dr. Anindita Borah , HOD, Department of Electronics and Communication Engineering, for the valuable guidance, encouragement, and technical insight provided throughout the duration of this project. Their support was instrumental in the successful completion of both the hardware implementation and the data-analysis phase of this work.

I extend my thanks to the faculty members and laboratory staff of the department for their assistance with hardware components and technical clarifications during the development and testing of the embedded system.

Bishnujyoti Gogoi

ABSTRACT

Indoor air quality is an important determinant of occupant health, comfort, and productivity, yet it is rarely monitored continuously in ordinary indoor spaces such as classrooms, laboratories, and homes. Prolonged exposure to poor ventilation, elevated humidity, and gaseous pollutants can affect wellbeing even when conditions are not immediately perceptible to occupants. This project presents the design and implementation of a low-cost Indoor Air Quality and Environmental Data Logger built around the ESP8266 NodeMCU microcontroller, intended to continuously acquire environmental parameters, log them for long-term analysis, and demonstrate a basic machine-learning-based interpretation of the collected data.

The hardware system integrates a DHT11 sensor for temperature and relative humidity measurement, an MQ-135 gas sensor for relative air-quality sensing through its raw analog output, and a digital-output LDR module for ambient light sensing. The ESP8266 periodically samples these sensors, classifies the air quality into Good, Moderate, or Poor categories using fixed raw-ADC thresholds, flags a binary gas-anomaly condition when the MQ-135 reading exceeds a configured threshold, and derives an approximate occupancy status from the light sensor state. The processed readings are transmitted in real time to the Blynk IoT platform for live dashboard visualization and are simultaneously logged to a Google Sheets spreadsheet through a Google Apps Script web application, from which a CSV dataset was exported for offline analysis.

The exported dataset, comprising 1,499 timestamped readings collected over approximately 28 hours, was analyzed using Python within a Google Colab environment. Exploratory data analysis was carried out using Pandas, NumPy, Matplotlib, and Seaborn to examine temporal trends, class distributions, and correlations among the recorded parameters. For the machine-learning component, the features were encoded and standardized using Scikit-learn, and an Artificial Neural Network was constructed using TensorFlow/Keras to classify the binary Gas Anomaly condition from the six standardized input features. The network, consisting of two hidden dense layers of 64 and 32 neurons with ReLU activation and a sigmoid output layer, was trained for 50 epochs and evaluated on a held-out 20% test split, achieving a test accuracy of 100% and a precision score of 1.0. A parallel data-preparation pipeline for classifying the three-class Air Quality label was also prepared but not carried through to a trained model within the current notebook.

The results demonstrate that a low-cost, microcontroller-based sensing platform can generate a usable environmental dataset and that simple neural architectures can recover the deterministic threshold-based labelling rules embedded in the firmware with very high accuracy. The report also discusses the limitations of the uncalibrated gas sensor, the class imbalance present in the collected dataset, and the fact that the near-perfect classification accuracy reflects the rule-based nature of the ground-truth labels rather than a complex learned relationship. The system provides a practical, extensible foundation for future work involving calibrated sensors, larger and more diverse datasets, and more sophisticated predictive models.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AQI	Air Quality Index
IoT	Internet of Things
ML	Machine Learning
ANN	Artificial Neural Network
ADC	Analog-to-Digital Converter
GPIO	General Purpose Input/Output
CSV	Comma-Separated Values
DHT	Digital Humidity and Temperature (sensor)
IAQ	Indoor Air Quality
LDR	Light Dependent Resistor
Wi-Fi	Wireless Fidelity
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
ReLU	Rectified Linear Unit
EDA	Exploratory Data Analysis

CHAPTER 1: INTRODUCTION

1.1 Background

Air quality and environmental comfort inside enclosed spaces directly influence human health, concentration, and general wellbeing. Unlike outdoor air quality, which is monitored extensively by government and municipal agencies, indoor environments such as classrooms, offices, laboratories, and residential rooms are seldom monitored continuously. Parameters such as temperature, relative humidity, and the presence of volatile gases can vary significantly over the course of a day due to occupancy, ventilation conditions, and external weather, yet these variations often go unrecorded in ordinary settings.

1.2 Importance of Indoor Air Quality

Indoor Air Quality (IAQ) refers to the condition of air within and around buildings, particularly as it relates to the health and comfort of occupants. Elevated humidity can promote microbial growth, uncomfortable temperatures can reduce concentration and productivity, and the accumulation of gases from combustion, cleaning agents, or poor ventilation can pose long-term health risks. Continuous monitoring of these parameters allows early detection of unfavourable conditions and can inform corrective action such as improved ventilation.

1.3 Environmental Data Logging

Environmental data logging refers to the automated, periodic recording of environmental parameters over time. Unlike a single instantaneous measurement, a logged dataset captures temporal trends, allowing patterns such as daily temperature cycles, occupancy-related fluctuations, and gradual air-quality degradation to be identified. Such datasets are also a prerequisite for applying data-driven and machine-learning techniques, since a sufficiently large and time-stamped record of observations is required to train and evaluate predictive models.

1.4 Problem Statement

Conventional indoor environments generally lack any continuous, low-cost mechanism for tracking temperature, humidity, and gas-related air-quality indicators over time. Where monitoring does exist, it is often confined to a single real-time display without a

corresponding historical dataset that could be used for further analysis. This project addresses the need for a low-cost, microcontroller-based system that can continuously acquire environmental readings, present them in real time, log them for long-term storage, and generate a structured dataset suitable for subsequent data analysis and machine-learning experimentation.

1.5 Motivation

Commercial indoor air-quality monitoring systems with calibrated sensors and cloud dashboards are often costly and are not easily accessible for academic experimentation or small-scale deployment. A system built from widely available modules such as the ESP8266 NodeMCU, a DHT-series temperature and humidity sensor, and a low-cost MQ-series gas sensor demonstrates that a functional environmental logging and analysis pipeline can be constructed with modest hardware, while also serving as a practical platform for learning embedded systems, IoT data transmission, and applied machine learning.

1.6 Objectives

- To design and implement a microcontroller-based environmental monitoring system using the ESP8266 NodeMCU.
- To acquire temperature and humidity readings using a DHT11 sensor.
- To acquire a relative, uncalibrated air-quality indication using an MQ-135 gas sensor.
- To sense ambient light/occupancy-related conditions using an LDR module.
- To classify air quality into Good, Moderate, and Poor categories using fixed raw-ADC thresholds implemented in firmware.
- To flag a binary gas-anomaly condition when the MQ-135 reading exceeds a configured threshold.
- To estimate an approximate occupancy status from the LDR digital state.
- To transmit live readings to the Blynk IoT platform for real-time visualization.
- To log all readings to a Google Sheets spreadsheet via a Google Apps Script web application for long-term storage.

- To export the logged data as a CSV dataset and perform exploratory data analysis using Python.
- To build and evaluate an Artificial Neural Network for classifying the Gas Anomaly condition from the recorded environmental parameters.

1.7 Scope

The scope of this project covers the hardware design and firmware implementation of the sensing and data-transmission unit, the logging pipeline to Blynk and Google Sheets, and the offline data analysis and neural-network-based classification of the exported dataset. The project does not include professional calibration of the gas sensor against certified reference instruments, and the dataset used for analysis was collected from a single location over a limited time window. The project also does not implement occupancy detection using dedicated presence sensors (e.g., PIR) or machine-learning-based occupancy prediction; occupancy status is only an approximate estimate derived from the light sensor.

1.8 Applications

- Continuous environmental monitoring in classrooms and laboratories.
- Low-cost indoor air-quality awareness in homes and offices.
- Generation of environmental datasets for academic research and machine-learning experimentation.
- A demonstrative platform for teaching IoT-based data acquisition and cloud logging.
- A foundation for future smart-building and automated-ventilation applications.

1.9 Organization of Report

Chapter 2 reviews the relevant background theory of indoor air quality, the sensors used, and the microcontroller platform. Chapter 3 describes the overall system design and methodology. Chapter 4 details the hardware implementation, including the exact pin connections used in the firmware. Chapter 5 explains the software implementation on both the embedded and Python sides. Chapter 6 presents the collected dataset and the exploratory data analysis performed on it. Chapter 7 discusses the machine-learning aspects of the project, and Chapter 8 presents the Artificial Neural Network implementation and its

results in detail. Chapter 9 consolidates the results and discussion, Chapter 10 discusses advantages, limitations, and applications, Chapter 11 outlines future scope, and Chapter 12 concludes the report. Appendices provide the complete firmware source code, key Python code, and a sample of the collected dataset.

CHAPTER 2: LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 Introduction to Indoor Air Quality

Indoor Air Quality is influenced by a combination of ventilation rate, occupancy, building materials, external weather conditions, and the presence of pollutant sources such as combustion appliances, cleaning products, or biological contaminants. Standard indicators used to describe IAQ include temperature, relative humidity, carbon dioxide concentration, particulate matter, and concentrations of volatile organic compounds and other gases.

2.2 Factors Affecting Indoor Air Quality

- Ventilation rate and air exchange with the outdoor environment.
- Occupancy density and human respiration.
- Temperature and humidity levels, which affect comfort and microbial growth.
- Presence of combustion sources, cleaning agents, or off-gassing materials.
- Building design, including window placement and HVAC operation.

2.3 Temperature and Humidity Monitoring

Temperature and relative humidity are two of the most fundamental environmental parameters. High humidity combined with warm temperatures can encourage mould and dust-mite growth, while low humidity can cause discomfort and respiratory irritation. Continuous monitoring of these two parameters therefore forms the baseline of most environmental monitoring systems, including the one implemented in this project.

2.4 Gas and Air-Quality Sensors

The MQ-135 is a low-cost metal-oxide semiconductor (MOS) gas sensor commonly used for general air-quality sensing applications. It uses a tin dioxide (SnO2) sensing layer whose electrical conductivity changes in the presence of reducing gases such as ammonia, benzene, alcohol vapours, smoke, and carbon dioxide. The sensor includes an integrated heating element that must reach a stable operating temperature before readings become consistent, and it produces an analog voltage output that is proportional to the concentration of gases present, but is not itself calibrated to a specific gas or to parts-per-million (ppm) units.

Because the MQ-135 responds to a broad mixture of gases with overlapping sensitivity (cross-sensitivity), and because no reference-gas calibration was performed in this project, the sensor's analog output is used strictly as a raw, relative indicator of air quality rather than as a calibrated pollutant concentration. This distinction is maintained consistently throughout this report.

2.5 Temperature and Humidity Sensor (DHT11)

The DHT11 is a low-cost digital temperature and humidity sensor that combines a resistive humidity-sensing element and a thermistor, coupled with an internal analog-to-digital converter that outputs a calibrated digital signal over a single-wire interface. It has a typical humidity measurement range of approximately 20% to 90% RH with a nominal accuracy of about ±5% RH, and a temperature range of approximately 0°C to 50°C with an accuracy of about ±2°C. Its relatively coarse resolution and slower sampling rate make it suitable for general environmental monitoring rather than precision measurement.

2.6 Light Detection (LDR)

A Light Dependent Resistor (LDR) is a photo resistive component whose resistance decreases as incident light intensity increases. In this project, the LDR is used through a comparator-based LDR module that produces a digital HIGH/LOW output depending on whether the ambient light level is above or below a threshold set by an onboard potentiometer, rather than a calibrated lux measurement. The digital light state is used in the firmware as an indirect proxy for probable room occupancy, on the basis that a lit room is more likely to be occupied than a dark one; this is an approximate heuristic and not a validated occupancy-detection technique.

2.7 Microcontroller Platform (ESP8266 NodeMCU)

The firmware in this project targets the ESP8266 platform, as confirmed by the inclusion of the ESP8266WiFi.h, ESP8266HTTPClient.h, and BlynkSimpleEsp8266.h libraries in the source code. The ESP8266 is a low-cost System-on-Chip featuring an integrated 802.11 b/g/n Wi-Fi transceiver and a Tensilica L106 32-bit RISC processor, commonly used in IoT applications through development boards such as the NodeMCU. The NodeMCU board exposes GPIO pins, a single analog input channel (A0), and provides onboard voltage regulation and USB-to-serial conversion for programming, which the firmware uses to read the MQ-135 analog output and to drive the digital sensor interfaces.

2.8 Data Logging

Two complementary data-logging approaches are used in this project: a real-time IoT dashboard (Blynk) for immediate visualization, and a persistent spreadsheet-based log (Google Sheets, accessed through a Google Apps Script web application) for long-term storage and subsequent export as a CSV dataset. This dual approach allows the system to serve both as a live monitor and as a dataset-generation tool.

2.9 Machine Learning in Environmental Monitoring

Machine learning techniques are increasingly applied to environmental datasets to classify conditions, detect anomalies, or predict future states from historical sensor readings. Supervised classification algorithms learn a mapping between input features (such as temperature, humidity, and gas-sensor readings) and a labelled target variable (such as an air-quality category or an anomaly flag), using a portion of the collected data for training and a held-out portion for evaluation.

2.10 Artificial Neural Networks

An Artificial Neural Network (ANN) is a machine-learning model composed of layers of interconnected nodes (neurons), each applying a weighted sum of its inputs followed by a non-linear activation function such as the Rectified Linear Unit (ReLU) or the sigmoid function. Multi-layer feed-forward ANNs, such as the one implemented in this project using TensorFlow/Keras, are capable of learning non-linear relationships between input features and a target variable through iterative training using gradient-based optimization (e.g., the Adam optimizer) and a loss function appropriate to the task, such as binary cross-entropy for binary classification.

2.11 Research Gap / Motivation for Proposed System

While numerous commercial and research-grade indoor air-quality monitoring systems exist, many rely on calibrated, industrial-grade sensors that are unsuitable for low-budget academic prototyping. There is a practical gap for simple, reproducible systems built from widely available modules that can still generate a usable dataset for exploratory data analysis and introductory machine-learning experimentation. The system presented in this report is intended to fill that gap at a prototype/academic level, while being transparent about the limitations introduced by the use of uncalibrated, low-cost sensors.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Proposed System

The proposed system consists of an ESP8266 NodeMCU microcontroller interfaced with a DHT11 temperature-and-humidity sensor, an MQ-135 gas sensor, and a digital-output LDR module. The microcontroller samples all three sensors at a fixed interval, performs simple threshold-based classification of the air quality and gas-anomaly condition, estimates an approximate occupancy status, and transmits the resulting record simultaneously to the Blynk IoT platform (for real-time visualization) and to a Google Sheets spreadsheet (for long-term storage). The stored data was subsequently exported as a CSV file and analyzed offline using Python in a Google Colab environment, where exploratory data analysis and an ANN-based classification model were implemented.

3.2 System Architecture

The overall architecture follows a linear pipeline from sensing to analysis, illustrated in Figure 3.1.

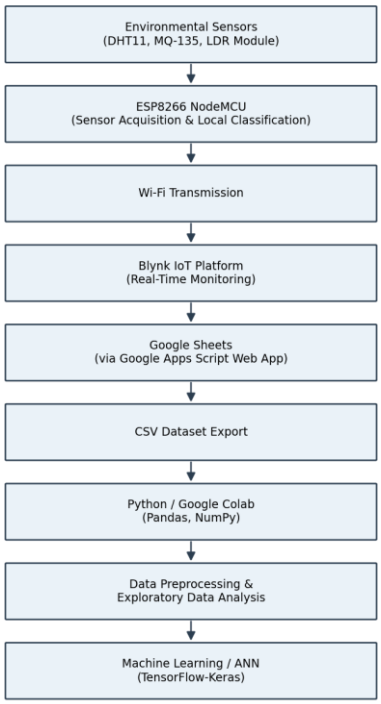

Figure 3.1: System Block Diagram

3.3 Working Principle

- 1. The DHT11 sensor is read to obtain the current temperature and humidity; readings are validated to reject NaN results.
- 2. The MQ-135 analog output is read via the ESP8266's onboard ADC (pin A0), yielding a raw, uncalibrated value.
- 3. The LDR module's digital output is read to obtain the current light state (HIGH/LOW).
- 4. The firmware classifies air quality into Good, Moderate, or Poor based on fixed thresholds applied to the raw MQ-135 reading.
- 5. The firmware flags a binary Gas Anomaly condition if the raw MQ-135 reading exceeds a configured threshold (600).
- 6. The firmware estimates an Occupancy Status (“Unoccupied” or “PossiblyOccupied”) from the LDR digital state.
- 7. All readings and derived values are transmitted to the Blynk dashboard for real-time viewing, when connected.
- 8. The same record is sent as an HTTPS GET request to a Google Apps Script web application, which appends it to a Google Sheets spreadsheet.
- 9. This cycle repeats at a fixed sampling interval, and the accumulated spreadsheet data was exported as a CSV file for offline analysis.
- 10. In Python, the CSV dataset was loaded, preprocessed, and explored using descriptive statistics and visualizations.
- 11. An Artificial Neural Network was trained on the standardized features to classify the Gas Anomaly condition.

3.4 System Flowchart

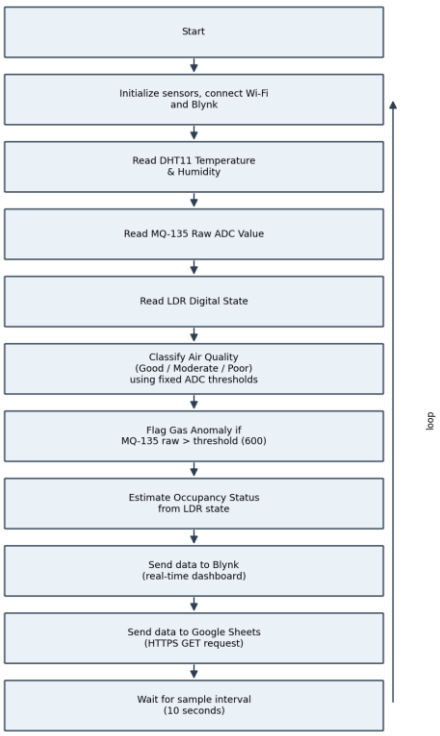

Figure 3.2: System Working Flowchart

3.5 Hardware Methodology

The hardware methodology involved selecting a single ESP8266 NodeMCU development board as the central controller, wiring the DHT11, MQ-135, and LDR module to the pins defined in the firmware, and powering the assembly from the NodeMCU's onboard 3.3 V regulator (with the MQ-135 heater typically supplied from the board's 5 V/VIN pin per standard module wiring). Full wiring details, as extracted from the firmware, are presented in Chapter 4.

3.6 Software Methodology

The software methodology spans two environments: embedded C/C++ firmware written in the Arduino IDE for the ESP8266, and Python code executed in Google Colab for offline data analysis. The firmware handles sensor acquisition, threshold-based classification, and dual-channel transmission (Blynk and Google Sheets). The Python pipeline handles data loading, preprocessing, exploratory analysis, and neural-network-based classification, using Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, and TensorFlow/Keras.

3.7 Data Acquisition Method

Sensor data is acquired periodically by a BlynkTimer callback configured with an interval of 10,000 milliseconds (10 seconds), as defined by the sampleIntervalMs variable in the firmware. Each acquisition cycle reads all three sensors, computes the derived classification values, and immediately attempts transmission to both Blynk and Google Sheets.

3.8 Data Storage Method

Two storage mechanisms are used: Blynk's cloud-hosted virtual pins provide transient, real-time storage suitable for live dashboard display, while the Google Sheets spreadsheet (populated via a Google Apps Script web application) provides a persistent, append-only log of every transmitted reading. The Google Sheets log is the source from which the CSV dataset used for analysis in this report was exported.

3.9 Data Analysis Methodology

The exported CSV dataset was loaded into a Pandas DataFrame in Google Colab. Basic data-quality checks (null-value counts, data types) were performed, followed by exploratory visualizations of temporal trends, class distributions, and correlations among the numerical parameters. Categorical/text columns (Timestamp, Air quality, Occupancy Status) were transformed as needed for the machine-learning stage described below.

3.10 Machine-Learning Methodology

For the machine-learning stage, the Timestamp column was split into separately label-encoded Date and Time features, and the Occupancy Status column was dropped from the machine-learning dataset. Six features — Temperature, Humidity, MQ-135 raw value, Light State, encoded Date, and encoded Time — were standardized using Scikit-learn's StandardScaler. An 80:20 stratified train–test split (random_state = 42) was used. An Artificial Neural Network was then built, trained, and evaluated to classify the binary Gas Anomaly target from these six standardized features; this is described in detail in Chapter 8. A parallel feature/target split was also prepared for classifying the three-class Air Quality label, but no trained model or evaluation results for that task are present in the analyzed notebook [INFORMATION TO BE ADDED if a model was trained subsequently].

CHAPTER 4: HARDWARE IMPLEMENTATION

4.1 Hardware Components

Table 4.1 lists the hardware components confirmed by the firmware source code (AQI_sensor.ino).

Table 4.1: Hardware Components Used

Sl. No.	Component	Quantity	Purpose
1	ESP8266 NodeMCU Development Board	1	Main microcontroller: sensor acquisition, classification logic, Wi-Fi/HTTPS transmission
2	DHT11 Temperature & Humidity Sensor	1	Measures ambient temperature and relative humidity
3	MQ-135 Gas Sensor Module	1	Provides a raw, uncalibrated analog indication of air quality/gas concentration
4	LDR Module (digital output, comparator-based)	1	Provides a digital light-state signal used as an occupancy proxy
5	Jumper wires / breadboard	As required	Circuit interconnection
6	USB cable / power supply	1	Power and programming of the ESP8266 NodeMCU

4.2 ESP8266 NodeMCU Microcontroller

The firmware includes ESP8266WiFi.h, ESP8266HTTPClient.h, WiFiClientSecure.h, and BlynkSimpleEsp8266.h, confirming that the target platform is the ESP8266 (NodeMCU form factor), and not an ESP32. The ESP8266 provides a single analog input channel used here for the MQ-135, and multiple digital GPIO pins used for the DHT11 and LDR interfaces, along with integrated 802.11 b/g/n Wi-Fi used for both the Blynk connection and the HTTPS request to Google Apps Script.

4.3 Gas Sensor (MQ-135)

The MQ-135 is connected to the analog input pin A0 of the NodeMCU. The firmware reads this pin using analogRead(), producing a raw ADC value with no onboard conversion to ppm or any calibrated gas-concentration unit. This raw value is used directly for the air-quality classification and gas-anomaly logic described in Section 4.9.

4.4 Temperature/Humidity Sensor (DHT11)

The DHT11 is connected to digital pin D4 and is read using the Adafruit-style DHT library (DHT.h), via dht.readTemperature() and dht.readHumidity(). The firmware checks both readings with isnan() and skips the logging cycle if either read fails, providing basic error handling against transient sensor faults.

4.5 LDR Module

The LDR module's digital output is connected to pin D5 and configured as an input using pinMode(LDRPIN, INPUT). The firmware reads this pin with digitalRead() to obtain a binary light state, which is then used to derive the Occupancy Status field.

4.6 Additional Sensors

No additional sensors (such as a dedicated particulate-matter sensor, CO2 sensor, or PIR motion sensor) are present in the uploaded firmware. [INFORMATION TO BE ADDED if additional sensors were used in a hardware revision not reflected in this source file.]

4.7 Circuit Connections

Table 4.2 lists the exact pin assignments extracted from the #define statements and pinMode()/dht() calls in AQI_sensor.ino.

Table 4.2: Sensor-to-ESP8266 Pin Connections

Component	Sensor Pin	ESP8266 Pin	Purpose
DHT11	Data pin	D4	Temperature and humidity acquisition (single-wire digital interface)
MQ-135	Analog output (AOUT)	A0	Raw analog gas-sensor reading (ADC input)
LDR Module	Digital output (DOUT)	D5	Digital light-state input

Power connections (VCC and GND for each module) are not explicitly defined in the source code, as they are wired directly to the NodeMCU's 3V3/5V and GND pins rather than controlled by firmware. [INFORMATION TO BE ADDED: exact power-pin wiring, if documented separately.]

4.8 Complete Circuit Description

The DHT11 data line is connected to GPIO D4 with the sensor's VCC and GND connected to the NodeMCU's 3.3 V and GND rails respectively (a pull-up resistor may be required

depending on the specific DHT11 breakout used). The MQ-135 module's analog output is connected to the NodeMCU's single analog pin A0, with its VCC typically connected to 5 V (VIN) to properly power the sensor's internal heating element, per standard MQ-135 module wiring, and GND to the common ground. The LDR module's digital output is connected to GPIO D5, with its onboard comparator threshold adjustable via its potentiometer, and its VCC/GND connected to the NodeMCU's 3.3 V and GND rails.

4.9 Hardware Working

On power-up, the firmware initializes the DHT11 and configures the LDR pin as an input, then attempts to connect to the configured Wi-Fi network and to the Blynk service. Once connected, a BlynkTimer triggers the sendSensorData() function every 10 seconds. Within this function, the DHT11, MQ-135, and LDR are read; the air-quality class, gas-anomaly flag, and occupancy status are computed from fixed thresholds; the values are printed to the serial monitor for debugging; and the record is transmitted to both Blynk (if connected) and Google Sheets (via an HTTPS GET request to the configured Apps Script URL, provided Wi-Fi is connected).

CHAPTER 5: SOFTWARE IMPLEMENTATION

5.1 Arduino IDE

The embedded firmware (AQI_sensor.ino) was developed for the Arduino IDE targeting the ESP8266 board package, which provides the ESP8266WiFi, ESP8266HTTPClient, and WiFiClientSecure libraries used in this project, along with the BlynkSimpleEsp8266 library for IoT connectivity and the DHT library for sensor access.

5.2 Embedded Program

The firmware begins by defining the Blynk template credentials, Wi-Fi credentials, and the Google Apps Script deployment URL as global constants (redacted in Appendix A of this report for security; the student's local copy retains the real values). Pin definitions (DHTPIN, MQ135PIN, LDRPIN), a DHT object, a BlynkTimer instance, the sampling interval (sampleIntervalMs = 10000), and the gas-anomaly threshold (gasAnomalyThreshold = 600) are declared as global variables.

The logToGoogleSheets() function checks the Wi-Fi connection status, configures a WiFiClientSecure with certificate verification disabled (client.setInsecure()) and an extended timeout, URL-encodes the classification strings, constructs a GET request URL with the sensor values as query parameters, and sends it to the configured Google Apps Script endpoint, printing the HTTP response code and payload to the serial monitor for debugging.

The sendSensorData() function reads all three sensors, validates the DHT11 reading, computes the air-quality classification, gas-anomaly flag, and occupancy status using the fixed thresholds described in Chapter 4, prints a formatted summary to the serial monitor, writes each value to its corresponding Blynk virtual pin (V0–V6) if connected, and finally calls logToGoogleSheets() to persist the record.

The setup() function initializes the serial port, starts the DHT11, configures the LDR pin, connects to Wi-Fi with a bounded retry loop (up to 30 attempts with 500 ms delays), configures and connects to Blynk, and registers sendSensorData() with the BlynkTimer at the configured sampling interval. The loop() function services the Blynk connection (reconnecting if necessary) and runs the timer, which invokes sendSensorData() at each

interval. No explicit error-recovery logic beyond the DHT NaN check and the Wi-Fi/Blynk reconnection attempts is present in the firmware.

5.3 Python Environment

The data-analysis notebook (AQI_DL.ipynb) was executed in Google Colab. The libraries actually used are Pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, Scikit-learn (LabelEncoder, StandardScaler, train_test_split, and the classification-report/confusion-matrix/accuracy/precision metrics) for preprocessing and evaluation, and TensorFlow/Keras (Sequential, Dense) for building and training the Artificial Neural Network. The notebook also imports PyTorch (torch, torch.nn, torchvision) and Pillow (PIL.Image); however, these libraries are not used in any of the executed cells and appear to originate from a boilerplate template rather than from the actual analysis performed.

5.4 Dataset Processing

The dataset was loaded directly from a file stored on Google Drive. The Occupancy Status column was dropped prior to the machine-learning stage, since it was not used as a feature or target in the analyzed notebook. The Timestamp column was converted to a datetime type and decomposed into separate Date and Time components, each of which was label-encoded into an integer representation using Scikit-learn's LabelEncoder.

5.5 Machine-Learning Implementation

A feature/target split was prepared for a three-class Air Quality classification task (Good/Moderate/Poor, label-encoded), using the same six standardized features described in Section 3.10. However, no classifier was trained or evaluated for this task within the analyzed notebook; only the data-preparation and split steps are present. [INFORMATION TO BE ADDED if a model was subsequently trained for this task.] No traditional machine-learning algorithms (e.g., Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, or Support Vector Machine) were implemented anywhere in the analyzed notebook.

5.6 Deep-Learning Implementation

A binary classification task predicting the Gas Anomaly label was fully implemented using an Artificial Neural Network built with TensorFlow/Keras. The same six standardized

features were used as input, an 80:20 stratified train–test split was applied, and the network was trained for 50 epochs with a batch size of 32 and a validation split of 0.2. The complete architecture, training procedure, and results are presented in Chapter 8.

CHAPTER 6: DATASET AND EXPLORATORY DATA ANALYSIS

6.1 Dataset Description

- Total rows: 1,499 (excluding the header row)
- Total columns: 8
- Time period: 19 July 2026, 00:17:35 to 20 July 2026, 04:25:56 (approximately 28 hours, with the record spacing indicating some gaps rather than perfectly continuous 10-second sampling)
- Missing values: none — all 1,499 rows have complete, non-null values across all 8 columns
- File format: single CSV file exported from the Google Sheets log

6.2 Dataset Attributes

Table 6.1: Dataset Attribute Description

Feature	Description	Data Type	Unit
Timestamp	Date and time of the reading	object (string, parsed to datetime)	date-time
Temperature(degree celcius)	Ambient temperature from the DHT11 sensor	float64	°C
Humidity	Relative humidity from the DHT11 sensor	int64	% RH
MQ-135 raw value	Raw ADC output of the MQ-135 gas sensor	int64	raw ADC counts (unitless, uncalibrated)
Light State	Digital output of the LDR module	int64 (0/1)	binary
Air quality	Firmware-assigned class: Good / Moderate / Poor	object (categorical)	—
Gas Anomaly	Firmware-assigned binary anomaly flag	int64 (0/1)	binary
Occupancy Status	Firmware-estimated status: Unoccupied / PossiblyOccupied	object (categorical)	—

6.3 Data Preprocessing

The following preprocessing operations were actually performed in the analyzed notebook prior to model training:

- The Occupancy Status column was dropped, since it was not used in the machine-learning stage.
- The Timestamp column was converted from string to a datetime type and split into separate Date and Time components.
- The Date and Time components were label-encoded into integers using Scikit-learn's LabelEncoder.
- The Air quality categorical column was label-encoded into an integer target for the (unfinished) Air Quality classification task.
- The six numerical/encoded features (Temperature, Humidity, MQ-135 raw value, Light State, encoded Date, encoded Time) were standardized using StandardScaler.
- An 80:20 train–test split was applied, stratified by the target label, with random_state = 42.

No missing-value imputation was required, as the dataset contained no null values. No explicit outlier-removal step was performed in the notebook; however, the exploratory analysis in Section 6.5 identifies at least one anomalous reading (see Figure 6.1) that likely reflects a transient sensor glitch rather than a genuine environmental condition.

6.4 Descriptive Statistics

Table 6.2: Descriptive Statistics of Numerical Parameters

Parameter	Mean	Std. Dev.	Min	Max
Temperature (°C)	30.14	1.39	0.00	31.80
Humidity (%)	87.92	3.66	80.00	97.00
MQ-135 raw value	898.38	176.24	314.00	997.00

The minimum recorded temperature of 0.00 °C is inconsistent with the surrounding readings (which otherwise remain in the 27–32 °C range throughout the dataset) and most likely reflects a single transient DHT11 read glitch that was not caught by the firmware's isnan() check, rather than a genuine environmental temperature drop. This is noted as a data-quality limitation in Chapter 10.

6.5 Data Visualization

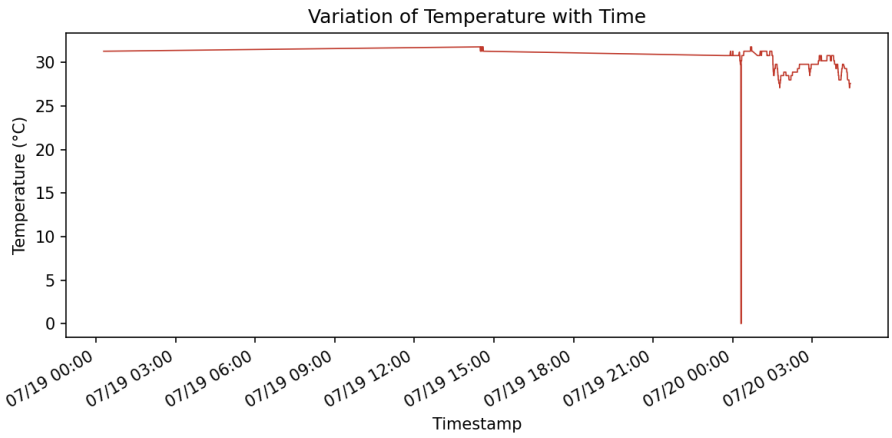

Figure 6.1: Variation of Temperature with Time

The temperature remained largely stable in the high-20s to low-30s (°C) range for most of the logging period, consistent with an indoor environment without strong external temperature swings. A single sharp dip to 0 °C is visible, corresponding to the anomalous reading discussed in Section 6.4.

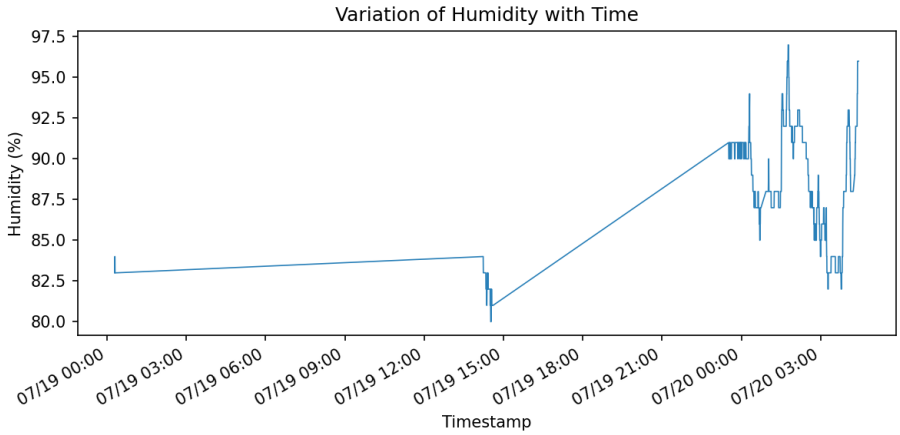

Figure 6.2: Variation of Humidity with Time

Relative humidity remained consistently high throughout the logging period (mostly in the 80–97% range), suggesting the measurement location experienced persistently humid indoor conditions during data collection.

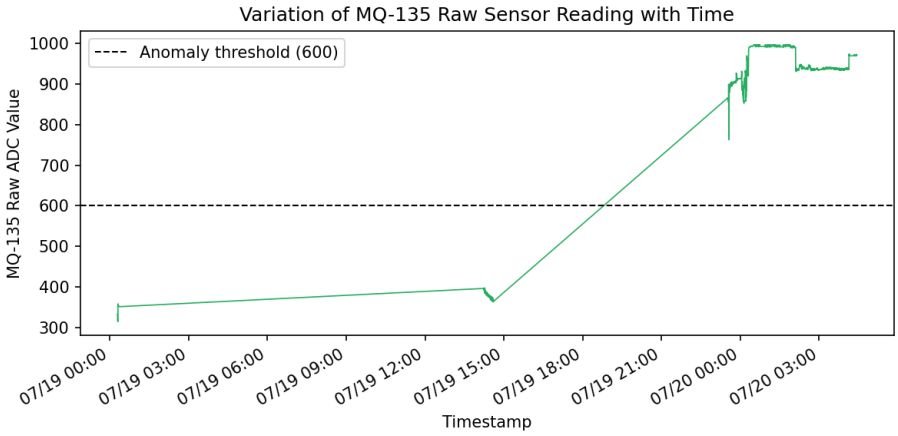

Figure 6.3: Variation of MQ-135 Raw Sensor Reading with Time

The raw MQ-135 output spent the great majority of the logging period above the firmware's gas-anomaly threshold of 600, with only a smaller number of intervals reading below it. This directly explains the class imbalance discussed below, since the Air Quality and Gas Anomaly labels are deterministic functions of this same raw value.

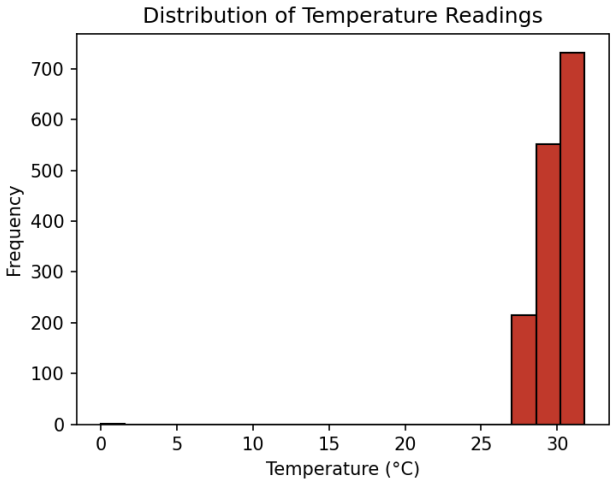

Figure 6.4: Distribution of Temperature Readings

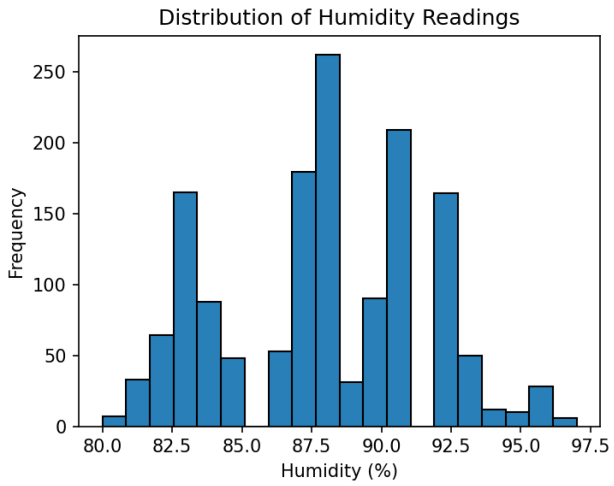

Figure 6.5: Distribution of Humidity Readings

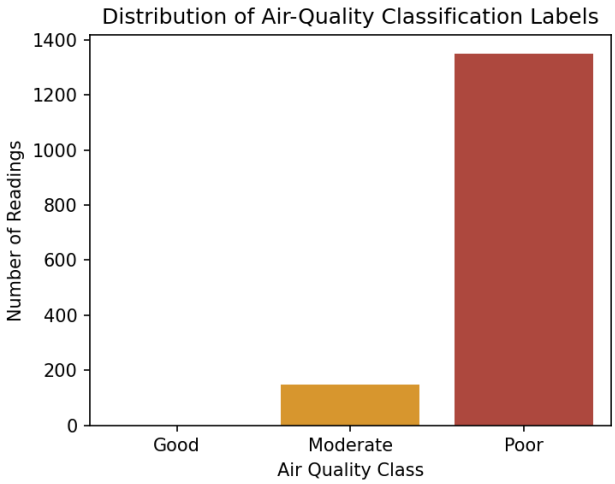

Figure 6.6: Distribution of Air-Quality Classification Labels

Of the 1,499 readings, 1,351 (approximately 90.1%) were classified as “Poor” and 148 (approximately 9.9%) as “Moderate”. No readings fell into the “Good” category during the logging window, indicating a significant class imbalance in the collected dataset and suggesting that the monitored location experienced consistently elevated MQ-135 readings throughout data collection.

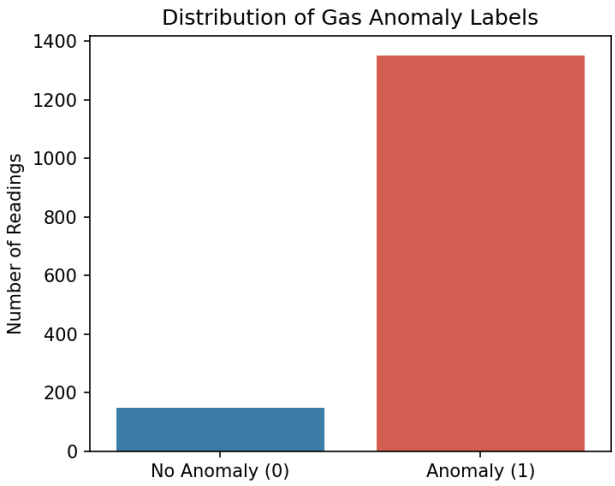

Figure 6.7: Distribution of Gas Anomaly Labels

The Gas Anomaly label shows the same approximately 90:10 imbalance as the Air Quality label, which is expected since both are derived from the same raw MQ-135 threshold logic in the firmware.

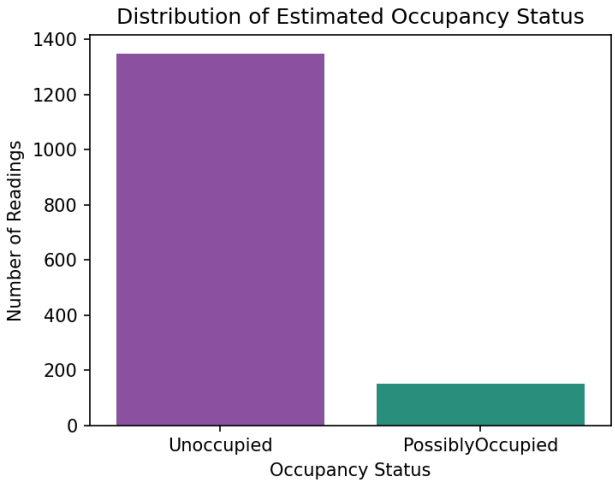

Figure 6.8: Distribution of Estimated Occupancy Status

The estimated Occupancy Status is “Unoccupied” for 1,349 readings (90.0%) and “PossiblyOccupied” for 150 readings (10.0%). As this status is derived purely from the LDR light state rather than any validated presence-detection method, it should be interpreted as an approximate light-based proxy rather than a confirmed occupancy record.

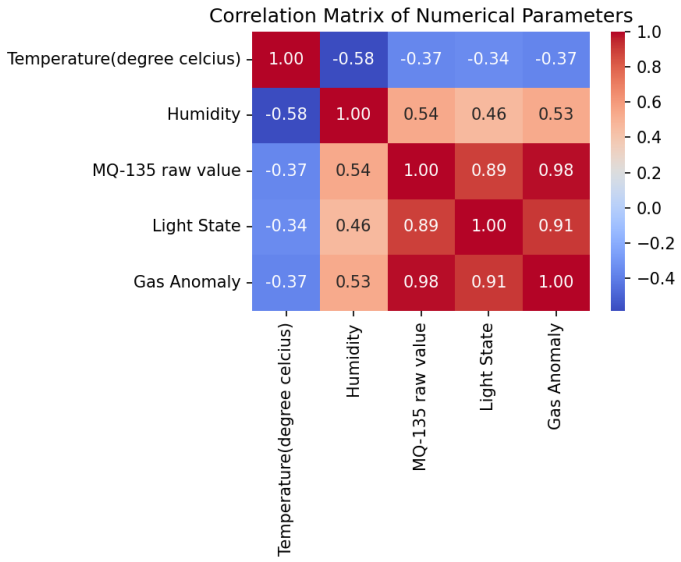

Figure 6.9: Correlation Matrix of Numerical Parameters

The correlation matrix quantifies the linear relationships among Temperature, Humidity, MQ-135 raw value, Light State, and Gas Anomaly. The Gas Anomaly column shows a strong relationship with the MQ-135 raw value by construction, since it is derived directly from that reading via a fixed threshold rather than being an independently measured quantity.

CHAPTER 7: MACHINE LEARNING ANALYSIS

7.1 Objective

Machine learning was applied to determine whether the environmental parameters recorded by the logger could be used to classify the derived Gas Anomaly and Air Quality labels. This chapter documents the feature/target preparation common to both tasks; the model actually trained and evaluated (an Artificial Neural Network for the Gas Anomaly task) is presented separately in Chapter 8, in line with the way the analysis was structured in the source notebook.

7.2 Feature Selection

Six input features were used for both prepared classification tasks: Temperature, Humidity, MQ-135 raw value, Light State, and the label-encoded Date and Time components derived from the Timestamp column.

7.3 Target Variable

Two target variables were prepared: the binary Gas Anomaly flag (0 = No Anomaly, 1 = Anomaly), for which a model was trained and evaluated, and the three-class Air quality label (Good/Moderate/Poor, label-encoded), for which the train–test split was prepared but no classifier was trained within the analyzed notebook.

7.4 Train–Test Split

An 80:20 train–test split was used for both tasks, with stratification on the respective target label and random_state = 42, giving 1,199 training samples and 300 test samples in each case.

7.5 Feature Scaling

Scikit-learn's StandardScaler was applied to the six input features (zero mean, unit variance) before model training, for both the Gas Anomaly and Air Quality preparation pipelines.

7.6 Classification

No traditional machine-learning classification algorithms — such as Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, or Support Vector Machine — were

implemented in the analyzed notebook. All classification work that reached a trained and evaluated model was carried out using an Artificial Neural Network, described in Chapter 8.

7.7 Regression Analysis

No regression task (e.g., predicting a continuous variable such as temperature or the raw MQ-135 value) was implemented in the analyzed notebook. This section is therefore not applicable to the current project. [INFORMATION TO BE ADDED if a regression task was implemented separately.]

7.8 Anomaly Detection

Gas anomaly detection in this project is implemented as a supervised binary classification problem rather than an unsupervised anomaly-detection technique: the ground-truth Gas Anomaly label is itself generated deterministically by a fixed threshold on the raw MQ-135 reading in the firmware (Section 4.9), and the ANN described in Chapter 8 was trained to recover this same rule from the recorded features.

7.9 Evaluation Metrics

Because only the ANN-based Gas Anomaly classifier reached a trained and evaluated state, its accuracy, precision, recall, F1-score, and confusion matrix are reported in Chapter 8 and consolidated in Chapter 9. No evaluation metrics are available for the Air Quality classification task, as no model was trained for it within the analyzed notebook. No regression evaluation metrics (MAE/MSE/RMSE/R^2) apply, since no regression task was implemented.

CHAPTER 8: ARTIFICIAL NEURAL NETWORK / DEEP LEARNING

8.1 Introduction

An Artificial Neural Network was selected to classify the binary Gas Anomaly condition because it can, in principle, learn non-linear decision boundaries directly from the standardized sensor features without requiring the modeller to hand-engineer the threshold rule. It should be noted, however, that since the ground-truth label is itself a simple threshold function of the MQ-135 raw value (one of the six input features), this is a comparatively easy learning task for the network, as reflected in the results reported below.

8.2 Input Features

The ANN takes six standardized input features: Temperature, Humidity, MQ-135 raw value, Light State, encoded Date, and encoded Time.

8.3 ANN Architecture

Table 8.1: ANN Architecture Summary (Gas Anomaly Model)

Layer	Neurons	Activation	Type
Input Layer	6 (matches number of input features)	—	Standardized feature vector
Hidden Layer 1	64	ReLU	Dense (fully connected)
Hidden Layer 2	32	ReLU	Dense (fully connected)
Output Layer	1	Sigmoid	Binary Gas Anomaly probability

Optimizer: Adam. Loss function: binary cross-entropy. Tracked metric: accuracy.

8.4 ANN Architecture Diagram

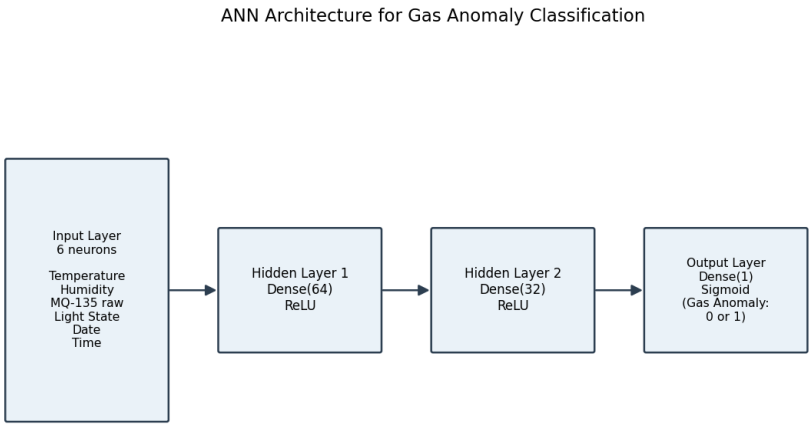

Figure 8.1: ANN Architecture for Gas Anomaly Classification

8.5 Training

- Epochs: 50
- Batch size: 32
- Validation split: 0.2 (taken from the training set)
- Training samples: 1,199; Test samples: 300 (80:20 stratified split)

8.6 Training and Validation Curves

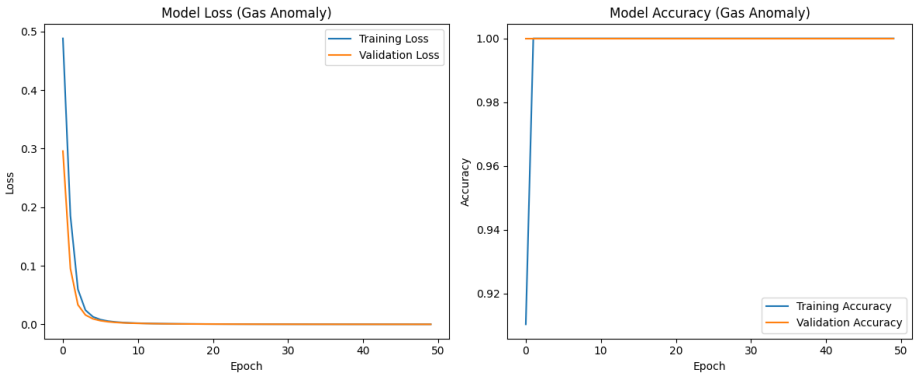

Figure 8.2: Training and Validation Loss/Accuracy Curves (Gas Anomaly ANN)

The training and validation loss curves both decrease rapidly within the first several epochs and remain low thereafter, while the corresponding accuracy curves rise quickly toward 1.0, indicating that the network converged with little difficulty on this task.

8.7 ANN Performance

On the held-out test set of 300 samples, the trained ANN achieved a test loss of 0.0001 and a test accuracy of 1.0000 (100%). The precision score on the test set was 1.0000.

8.8 Confusion Matrix

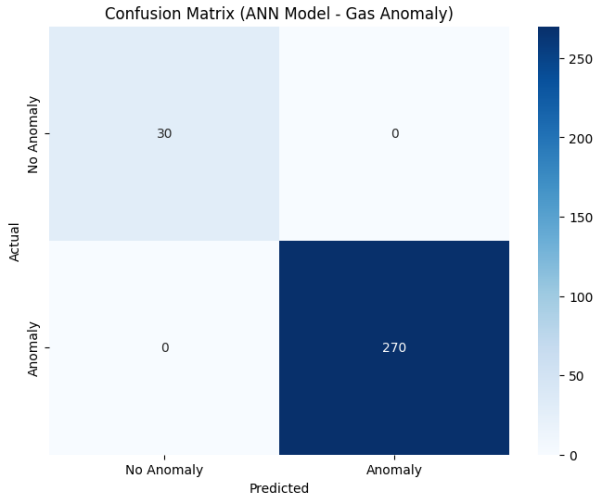

Figure 8.3: Confusion Matrix — ANN Gas Anomaly Model

The confusion matrix shows that all 30 “No Anomaly” test samples and all 270 “Anomaly” test samples were classified correctly, with zero false positives and zero false negatives.

8.9 Interpretation

The ANN achieved perfect classification accuracy on the Gas Anomaly task. This result should be interpreted with caution rather than as evidence of a highly complex learned relationship: the Gas Anomaly label is generated in the firmware by a deterministic threshold rule applied directly to the raw MQ-135 value, which is itself one of the six input features supplied to the network. In effect, the ANN was set the comparatively simple task of recovering a known threshold rule from a feature set that directly contains the thresholded quantity, rather than predicting an independently measured ground truth from indirect signals. The perfect test accuracy therefore primarily demonstrates that the network can reliably recover this deterministic rule, and it should not be read as a general measure of predictive difficulty for gas-anomaly detection in a setting where the anomaly label is independently and imperfectly measured. The pronounced class imbalance in the dataset (approximately 90% “Anomaly”) is also a relevant limitation, discussed further in Chapter 10.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Hardware Results

The ESP8266-based sensing system successfully acquired temperature, humidity, gas-sensor, and light-sensor readings and transmitted them over Wi-Fi to both the Blynk platform and Google Sheets for the duration of the logging period, producing a continuous dataset of 1,499 complete, non-null records spanning approximately 28 hours. This confirms that the basic hardware and connectivity design functioned as intended.

9.2 Environmental Data Results

Temperature readings remained in a relatively narrow band (mean 30.14 °C) apart from a single anomalous zero-value reading, and humidity remained consistently high (mean 87.92% RH) throughout the logging window, consistent with a warm, humid indoor environment during the data-collection period.

9.3 Air-Quality Results

Based on the firmware's fixed-threshold classification, 90.1% of readings were categorized as “Poor” and 9.9% as “Moderate” air quality, with no readings falling in the “Good” category during the observed period. As discussed in Chapter 2, these labels reflect an uncalibrated, relative MQ-135 reading rather than a validated pollutant concentration, and should be interpreted accordingly.

9.4 Occupancy Results

The LDR-based occupancy proxy indicated “PossiblyOccupied” conditions for 10.0% of readings and “Unoccupied” for the remaining 90.0%. No machine-learning model was applied to occupancy prediction in this project, since the Occupancy Status column was excluded from the machine-learning dataset.

9.5 Gas Anomaly Results

Gas Anomaly conditions (raw MQ-135 value above the configured threshold of 600) were flagged for 90.1% of readings, matching the Air Quality “Poor” proportion exactly, as both labels are derived from the same underlying threshold.

9.6 Machine-Learning Results

As noted in Chapter 7, no traditional machine-learning classifiers were implemented in the analyzed notebook, so no model-comparison table is presented for this section. [INFORMATION TO BE ADDED if traditional ML models are implemented in a future revision.]

9.7 ANN Results

Table 9.1: ANN Gas Anomaly Classification — Evaluation Metrics

Metric	Value
Accuracy	100.00%
Precision	1.0000
Test Loss (binary cross-entropy)	0.0001
Support (No Anomaly / Anomaly, test set)	30 / 270

9.8 Comparison of ML and ANN

A direct comparison between traditional machine-learning models and the ANN is not possible for this project, since no traditional ML classifier was trained on the same task. The ANN remains the only machine-learning model with completed training and evaluation results in the analyzed notebook.

9.9 Discussion

The overall system demonstrates a functioning, low-cost pipeline from sensor acquisition through to real-time dashboarding, persistent logging, and offline analysis. The near-perfect ANN accuracy on the Gas Anomaly task is best understood as confirmation that a small feed-forward network can reliably recover a known deterministic threshold rule already embedded in the firmware, rather than as evidence of a difficult predictive problem being solved. From an engineering standpoint, the more informative outcome of the data-analysis stage is the exploratory analysis itself — the observed class imbalance (approximately 90:10 in both Air Quality and Gas Anomaly labels), the consistently humid and moderately-to-poor-rated environment during the logging window, and the identification of a likely transient sensor-read glitch in the temperature series — all of which point to practical directions for improving both the hardware (calibration, sensor redundancy) and the dataset (longer collection across more varied conditions and locations) in future iterations of this project.

CHAPTER 10: ADVANTAGES, LIMITATIONS AND APPLICATIONS

10.1 Advantages

- Low overall hardware cost compared to commercial calibrated air-quality monitors.
- Continuous, unattended data logging without requiring manual readings.
- Dual-channel data delivery: a live dashboard (Blynk) alongside a persistent, exportable dataset (Google Sheets/CSV).
- The generated CSV dataset is directly compatible with standard Python data-analysis and machine-learning tooling.
- The firmware and sensor set are easy to replicate and extend with additional sensors or logic.

10.2 Limitations

- The MQ-135 sensor was not calibrated against a reference gas source, so its readings represent only a relative, uncalibrated indication of air quality rather than an accurate pollutant concentration in ppm.
- The MQ-135's sensing element responds to a broad range of gases with overlapping sensitivity (cross-sensitivity), so the raw reading cannot distinguish between specific pollutant types.
- ADC readings from a single low-cost analog sensor can vary with supply-voltage fluctuations and sensor ageing, which were not compensated for in this implementation.
- The DHT11 has comparatively coarse accuracy and resolution (±2 °C, ±5% RH) relative to higher-grade digital sensors.
- The occupancy status is only a light-based proxy and cannot reliably distinguish genuine human presence from ambient lighting changes unrelated to occupancy.
- The collected dataset spans only approximately 28 hours from a single location, limiting the generalizability of any patterns or trained models to other environments, seasons, or longer time frames.

- The dataset exhibits a strong class imbalance (approximately 90% “Poor”/“Anomaly” readings), which limits the ability to properly evaluate a classifier's behaviour on the minority class and can inflate apparent accuracy.
- The Gas Anomaly and Air Quality labels used for the ANN classification task are deterministic functions of one of the input features (the MQ-135 raw value) rather than independently measured ground truth, so the reported 100% ANN accuracy reflects successful recovery of a known rule rather than a demonstration of predictive skill on an independently verified target.
- The system depends on continuous Wi-Fi connectivity and on third-party cloud services (Blynk and Google Apps Script/Sheets) for both real-time display and data logging; an outage of either service interrupts a portion of the pipeline.

10.3 Applications

- Continuous indoor environmental monitoring in classrooms, laboratories, and offices.
- Low-cost awareness tool for humidity and relative air-quality conditions in homes.
- Educational platform for demonstrating IoT-based sensing, cloud logging, and applied machine learning.
- A foundation dataset-generation tool for further academic research into indoor-environment modelling.

CHAPTER 11: FUTURE SCOPE

The following improvements are proposed as realistic future extensions of this project and are not features currently implemented in the uploaded firmware or notebook:

- Integration of a calibrated or reference-checked gas sensor, or laboratory calibration of the existing MQ-135 against known gas concentrations.
- Addition of a dedicated particulate-matter sensor (e.g., PMS5003 or SDS011) for PM2.5/PM10 monitoring.
- Addition of a dedicated CO2 sensor (e.g., an NDIR-based module) for more specific indoor air-quality assessment.
- Collection of a substantially larger dataset spanning multiple days, seasons, and rooms to reduce class imbalance and improve generalizability.
- Deployment of multiple logging nodes across different rooms or buildings for comparative analysis.
- Development of a dedicated cloud dashboard or mobile application for more flexible remote monitoring, potentially replacing or supplementing Blynk.
- Use of MQTT or a dedicated IoT platform for more efficient and reliable data transmission than periodic HTTPS GET requests.
- Exploration of TinyML/edge-ML techniques to run lightweight inference directly on the ESP8266 or a more capable microcontroller.
- Implementation and comparison of traditional machine-learning classifiers (e.g., Logistic Regression, Random Forest) alongside the ANN, particularly on an independently measured occupancy or air-quality ground truth rather than firmware-generated threshold labels.
- Completion of the prepared Air Quality classification pipeline with a trained and evaluated model.
- Investigation of automated ventilation control triggered by real-time air-quality or occupancy predictions.

CHAPTER 12: CONCLUSION

This project set out to design and implement a low-cost Indoor Air Quality and Environmental Data Logger and to demonstrate a basic data-analysis and machine-learning pipeline on the data it produces. An ESP8266 NodeMCU microcontroller was interfaced with a DHT11 temperature-and-humidity sensor, an MQ-135 gas sensor, and a digital LDR module to periodically acquire environmental readings, classify air quality and gas-anomaly conditions using fixed firmware thresholds, and estimate an approximate occupancy status.

The system successfully transmitted readings in real time to the Blynk IoT platform and logged them persistently to a Google Sheets spreadsheet through a Google Apps Script web application, from which a dataset of 1,499 complete records spanning approximately 28 hours was exported and analyzed. Exploratory data analysis in Python revealed a consistently warm and humid indoor environment during the logging period, a pronounced class imbalance in the derived Air Quality and Gas Anomaly labels, and a likely transient sensor glitch in a single temperature reading.

An Artificial Neural Network with two hidden dense layers was built using TensorFlow/Keras and trained to classify the binary Gas Anomaly condition from the standardized sensor features, achieving 100% test accuracy. As discussed in Chapter 8, this result reflects the network's ability to recover the deterministic threshold rule already used to generate the label, rather than evidence of a complex predictive relationship being learned from independently verified ground truth. A parallel pipeline for classifying the three-class Air Quality label was prepared but not completed with a trained model in the analyzed notebook.

Overall, the project demonstrates that a functional, low-cost environmental-monitoring and dataset-generation pipeline can be built from widely available components and cloud services, and that the resulting dataset is suitable for introductory data-analysis and machine-learning exercises. The identified limitations — an uncalibrated gas sensor, a light-based occupancy proxy, a relatively short and imbalanced dataset, and firmware-derived (rather than independently measured) classification labels — provide clear and realistic directions for extending this work in future iterations.

REFERENCES

[1] Espressif Systems, “ESP8266EX Datasheet,” Espressif Systems, 2021.

[2] Aosong Electronics Co., Ltd., “DHT11 Humidity and Temperature Sensor Datasheet,” Aosong Electronics, 2014.

[3] Hanwei Electronics Co., Ltd., “MQ-135 Gas Sensor Technical Data,” Hanwei Electronics, n.d.

[4] Arduino, “Arduino Reference Documentation,” Arduino AG. [Online]. Available: https://www.arduino.cc/reference/en/

[5] Espressif Systems, “Arduino core for ESP8266 WiFi chip — Documentation,” GitHub Repository, esp8266/Arduino.

[6] Blynk Technologies Inc., “Blynk Documentation,” Blynk. [Online]. Available: https://docs.blynk.io/

[7] Google Developers, “Google Apps Script Documentation,” Google. [Online]. Available: https://developers.google.com/apps-script

[8] Python Software Foundation, “Python Language Reference,” Python Software Foundation. [Online]. Available: https://docs.python.org/3/

[9] W. McKinney, “Pandas: Python Data Analysis Library,” pandas development team. [Online]. Available: https://pandas.pydata.org/

[10] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” TensorFlow.org, 2015. [Online]. Available: https://www.tensorflow.org/

[12] F. Chollet, “Keras Documentation,” Keras.io. [Online]. Available: https://keras.io/

[13] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[14] M. L. Waskom, “seaborn: Statistical Data Visualization,” Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

[15] World Health Organization, “WHO Guidelines for Indoor Air Quality,” World Health Organization, Geneva.

Note: reference entries above are provided in standard IEEE style for known, verifiable sources (official documentation, datasheets, and widely cited software/library citations). Where an exact publication year, edition, or URL could not be verified for the specific hardware modules used, the entry has been kept general rather than inventing unverifiable details.

APPENDIX A: COMPLETE MICROCONTROLLER SOURCE CODE

The following is the complete firmware (AQI_sensor.ino) as uploaded, reproduced for reference. For security, the Blynk authentication token, Wi-Fi SSID/password, and the Google Apps Script deployment URL have been replaced with placeholder text; the student should retain the original credentials only in their own local copy of the code and must not publish real credentials in a submitted report.

```
/*
 * Indoor Air-Quality & Environmental Data Logger
 * ESP8266 NodeMCU + DHT11 + MQ-135 + LDR module
 * Sends data to Blynk (real-time) AND Google Sheets (long-term storage)
 */

// ============================================================
// BLYNK CREDENTIALS
// ============================================================
#define BLYNK_TEMPLATE_ID   "TMPL3syiQHI1Z"
#define BLYNK_TEMPLATE_NAME "AirQualityLogger"
#define BLYNK_AUTH_TOKEN    "<YOUR_BLYNK_AUTH_TOKEN>"

// ============================================================
// LIBRARIES
// ============================================================
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>
#include <BlynkSimpleEsp8266.h>
#include <DHT.h>

// ============================================================
// WIFI CREDENTIALS
// ============================================================
char ssid[] = "<YOUR_WIFI_SSID>";
char pass[] = "<YOUR_WIFI_PASSWORD>";

// ============================================================
// GOOGLE APPS SCRIPT WEB APP URL
// Replace YOUR_DEPLOYMENT_ID with your actual deployment ID
// ============================================================
String googleScriptURL =
  "<YOUR_GOOGLE_APPS_SCRIPT_DEPLOYMENT_URL>";

// ============================================================
// PIN SETUP
// ============================================================

// DHT11
#define DHTPIN D4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
```

```
// MQ-135 Analog Output
#define MQ135PIN A0

// LDR Module Digital Output
#define LDRPIN D5

// ========================================================
// TIMING
// ========================================================

BlynkTimer timer;

// Send/read data every 5 seconds
unsigned long sampleIntervalMs = 10000;

// ========================================================
// THRESHOLDS
// Tune these values after collecting real sensor data
// ========================================================

int gasAnomalyThreshold = 600;

// ========================================================
// GOOGLE SHEETS LOGGING FUNCTION
// ========================================================

void logToGoogleSheets(
  float t,
  float h,
  int mq,
  int light,
  String cls,
  int anomaly,
  String occupancy
) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi disconnected. Skipping Google Sheets.");
    return;
  }

  WiFiClientSecure client;

  // Disable certificate verification
  client.setInsecure();

  // Increase timeout for Google servers
  client.setTimeout(15000);

  HTTPClient https;

  // URL encode spaces in text values
  cls.replace(" ", "%20");
  occupancy.replace(" ", "%20");

  // Construct URL
  String url = googleScriptURL +
               "?t=" + String(t, 2) +
               "&h=" + String(h, 2) +
```

```
                "&mq=" + String(mq) +
                "&light=" + String(light) +
                "&cls=" + cls +
                "&anomaly=" + String(anomaly) +
                "&occupancy=" + occupancy;

  Serial.println();
  Serial.println("Sending data to Google Sheets...");
  Serial.print("Light: ");
  Serial.println(light);
  Serial.print("Occupancy: ");
  Serial.println(occupancy);

  // Follow Google redirects
  https.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

  // Increase HTTP timeout
  https.setTimeout(15000);

  if (https.begin(client, url)) {

    int httpCode = https.GET();

    Serial.print("Google Sheets HTTP status: ");
    Serial.println(httpCode);

    if (httpCode > 0) {

      String payload = https.getString();

      Serial.print("Google Sheets response: ");
      Serial.println(payload);

    } else {

      Serial.print("Google Sheets request failed: ");
      Serial.println(https.errorToString(httpCode));

    }

    https.end();

  } else {

    Serial.println("Unable to establish HTTPS connection.");

  }

  // Give ESP8266 WiFi stack a moment
  yield();
}
// =======================================================
// SENSOR READING + DATA SENDING FUNCTION
// =======================================================

void sendSensorData() {
```

```
// ------------------------------------------------
// Read DHT11
// ------------------------------------------------

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

if (isnan(temperature) || isnan(humidity)) {

  Serial.println("DHT11 read failed. Skipping this cycle.");

  return;
}

// ------------------------------------------------
// Read MQ-135
// ------------------------------------------------

int mq135Raw = analogRead(MQ135PIN);

// ------------------------------------------------
// Read LDR Digital Output
// ------------------------------------------------

int lightState = digitalRead(LDRPIN);

// ------------------------------------------------
// Air Quality Classification
// IMPORTANT:
// These are experimental RAW ADC thresholds.
// They do NOT represent calibrated PPM or AQI.
// ------------------------------------------------

String airQualityClass;

if (mq135Raw < 300) {

  airQualityClass = "Good";

} else if (mq135Raw < 600) {

  airQualityClass = "Moderate";

} else {

  airQualityClass = "Poor";

}

// ------------------------------------------------
// Gas Anomaly Detection
// ------------------------------------------------

int gasAnomaly;
```

```
if (mq135Raw > gasAnomalyThreshold) {

  gasAnomaly = 1;

} else {

  gasAnomaly = 0;

}

// ------------------------------------------------
// Simple Occupancy Estimation
//
// This is ONLY an estimated state based on light.
// LDR cannot reliably detect actual human occupancy.
//
// Change HIGH/LOW if your LDR module behaves opposite.
// ------------------------------------------------

String OccupancyStatus;

if (lightState == HIGH) {

  OccupancyStatus = "Unoccupied";

} else {

  OccupancyStatus = "PossiblyOccupied";

}

// ================================================
// SERIAL MONITOR OUTPUT
// ================================================

Serial.println("--------------------------------");

Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" C");

Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");

Serial.print("MQ135 Raw: ");
Serial.println(mq135Raw);

Serial.print("Light State: ");
Serial.println(lightState);

Serial.print("Air Quality: ");
Serial.println(airQualityClass);

Serial.print("Gas Anomaly: ");
```

```
  Serial.println(gasAnomaly);

  Serial.print("Occupancy Status: ");
  Serial.println(OccupancyStatus);

  Serial.println("--------------------------------");

  // ================================================
  // SEND DATA TO BLYNK
  // ================================================

  if (Blynk.connected()) {

    Blynk.virtualWrite(V0, temperature);
    Blynk.virtualWrite(V1, humidity);
    Blynk.virtualWrite(V2, mq135Raw);
    Blynk.virtualWrite(V3, lightState);
    Blynk.virtualWrite(V4, airQualityClass);
    Blynk.virtualWrite(V5, gasAnomaly);
    Blynk.virtualWrite(V6, OccupancyStatus);

  } else {

    Serial.println("Blynk not connected.");

  }

  // ================================================
  // SEND DATA TO GOOGLE SHEETS
  // ================================================

  logToGoogleSheets(
    temperature,
    humidity,
    mq135Raw,
    lightState,
    airQualityClass,
    gasAnomaly,
    OccupancyStatus
  );
}

// ========================================================
// SETUP
// ========================================================

void setup() {

  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("Indoor Air Quality Data Logger");
  Serial.println("Starting system...");

  // Start DHT11
```

```
  dht.begin();

  // Configure LDR pin
  pinMode(LDRPIN, INPUT);

  // ================================================
  // CONNECT TO WIFI
  // ================================================

WiFi.mode(WIFI_STA);
WiFi.disconnect();
delay(1000);

Serial.println("Starting WiFi connection...");
Serial.print("SSID: ");
Serial.println(ssid);

WiFi.begin(ssid, pass);

int attempts = 0;

while (WiFi.status() != WL_CONNECTED && attempts < 30) {
  delay(500);

  Serial.print("Attempt ");
  Serial.print(attempts);
  Serial.print(" | WiFi status: ");
  Serial.println(WiFi.status());

  attempts++;
}

if (WiFi.status() == WL_CONNECTED) {

  Serial.println();
  Serial.println("WiFi connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

} else {

  Serial.println();
  Serial.println("WiFi connection FAILED!");

  Serial.print("Final WiFi status code: ");
  Serial.println(WiFi.status());
}

  // ================================================
  // CONNECT TO BLYNK
  // ================================================

  Blynk.config(BLYNK_AUTH_TOKEN);

  Serial.println("Connecting to Blynk...");
```

```
  if (Blynk.connect(10000)) {

    Serial.println("Blynk connected!");

  } else {

    Serial.println("Blynk connection failed.");
  }

  // ================================================
  // START SENSOR TIMER
  // ================================================

  timer.setInterval(sampleIntervalMs, sendSensorData);

  Serial.println("System ready.");
}

// ========================================================
// MAIN LOOP
// ========================================================

void loop() {

  // Run Blynk only when connected
  if (Blynk.connected()) {

    Blynk.run();

  } else {

    // Attempt reconnection
    Blynk.connect(1000);
  }

  // Run sensor timer
  timer.run();
}
```

APPENDIX B: PYTHON / MACHINE-LEARNING CODE

The following key excerpts are reproduced from AQI_DL.ipynb, covering data preprocessing, feature scaling, and the ANN model used for Gas Anomaly classification (the only fully trained and evaluated model in the notebook).

B.1 Library Imports and Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow.keras as tf
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

seed = 42
random.seed(seed); np.random.seed(seed)
```

B.2 Data Loading and Preprocessing

```
data = pd.read_csv("<path to exported CSV dataset>")
data = data.drop(columns=['Occupancy Status'])

# Feature Encoding
from sklearn.preprocessing import LabelEncoder
data['Timestamp'] = pd.to_datetime(data['Timestamp'])
data['Date'] = data['Timestamp'].dt.date
data['Time'] = data['Timestamp'].dt.time

encoder = LabelEncoder()
data['Date'] = encoder.fit_transform(data['Date'])
data['Time'] = encoder.fit_transform(data['Time'])
```

B.3 Feature Scaling and Train–Test Split (Gas Anomaly Task)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

y_gas = data_gas_anomaly_task['Gas Anomaly'].copy()
features_for_scaling_gas = ['Temperature(degree celcius)', 'Humidity',
                            'MQ-135 raw value', 'Light State', 'Date', 'Time']

scaler_gas = StandardScaler()
X_gas =
scaler_gas.fit_transform(data_gas_anomaly_task[features_for_scaling_gas])

X_train_gas, X_test_gas, y_train_gas, y_test_gas = train_test_split(
    X_gas, y_gas, test_size=0.2, random_state=42, stratify=y_gas
)
```

B.4 ANN Model Definition and Training

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
input_dim_gas = X_train_gas.shape[1]

model_ann_gas = Sequential([
    Dense(64, activation='relu', input_shape=(input_dim_gas,)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_ann_gas.compile(optimizer='adam',
                      loss='binary_crossentropy',
                      metrics=['accuracy'])

history_gas = model_ann_gas.fit(
    X_train_gas, y_train_gas,
    epochs=50, batch_size=32, validation_split=0.2, verbose=0
)
```

B.5 Model Evaluation

```
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, precision_score

loss_gas, accuracy_ann_gas = model_ann_gas.evaluate(X_test_gas, y_test_gas,
verbose=0)

y_pred_gas_probs = model_ann_gas.predict(X_test_gas)
y_pred_gas = (y_pred_gas_probs > 0.5).astype(int)

accuracy_percentage = accuracy_score(y_test_gas, y_pred_gas) * 100
precision = precision_score(y_test_gas, y_pred_gas, zero_division=0)

print(classification_report(y_test_gas, y_pred_gas,
      target_names=['No Anomaly', 'Anomaly'], zero_division=0))
```

APPENDIX C: DATASET SAMPLE

Table C.1 shows the first 20 representative rows of the actual collected dataset (Readings_-_Sheet1.csv), reproduced as uploaded.

Table C.1: Sample of Collected Dataset (first 20 rows)

Timestamp	Temp(°C)	Humidity	MQ-135	Light	Air Quality	Anomaly	Occupancy
7/19/2026 0:17:35	31.3	83	333	0	Moderate	0	PossiblyOccupied
7/19/2026 0:17:45	31.3	83	330	0	Moderate	0	PossiblyOccupied
7/19/2026 0:17:55	31.3	83	320	0	Moderate	0	PossiblyOccupied
7/19/2026 0:18:05	31.3	84	319	1	Moderate	0	Unoccupied
7/19/2026 0:18:16	31.3	83	319	1	Moderate	0	Unoccupied
7/19/2026 0:18:26	31.3	83	314	1	Moderate	0	Unoccupied
7/19/2026 0:18:34	31.3	83	344	1	Moderate	0	Unoccupied
7/19/2026 0:18:45	31.3	83	351	1	Moderate	0	Unoccupied
7/19/2026 0:18:55	31.3	83	355	1	Moderate	0	Unoccupied
7/19/2026 0:19:05	31.3	83	358	1	Moderate	0	Unoccupied
7/19/2026 0:19:15	31.3	83	353	1	Moderate	0	Unoccupied
7/19/2026 0:19:25	31.3	83	354	1	Moderate	0	Unoccupied
7/19/2026 0:19:37	31.3	83	354	1	Moderate	0	Unoccupied
7/19/2026 0:21:49	31.3	83	351	1	Moderate	0	Unoccupied
7/19/2026 14:14:02	31.8	84	396	0	Moderate	0	PossiblyOccupied
7/19/2026 14:14:11	31.8	83	397	0	Moderate	0	PossiblyOccupied

Timestamp	Temp(°C)	Humidity	MQ-135	Light	Air Quality	Anomaly	Occupancy
7/19/2026 14:14:22	31.8	83	397	0	Moderate	0	PossiblyOccupied
7/19/2026 14:14:31	31.8	83	392	0	Moderate	0	PossiblyOccupied
7/19/2026 14:14:42	31.8	83	391	0	Moderate	0	PossiblyOccupied
7/19/2026 14:14:51	31.8	83	391	0	Moderate	0	PossiblyOccupied

Note the irregular spacing between some consecutive timestamps (e.g., a gap from 0:21:49 to 14:14:02 on 19 July), indicating that the logger was not continuously powered/connected for the entire monitoring window. This is documented as a limitation in Chapter 10.

APPENDIX D: ADDITIONAL RESULTS

The additional confusion-matrix renderings below were produced by repeated executions of the evaluation cell in the analyzed notebook and are included here for completeness; they are consistent with the primary result reported in Figure 8.3.

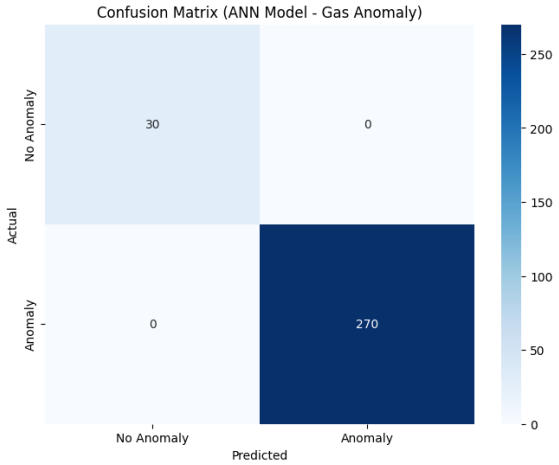

Figure D.1: Additional Confusion Matrix Rendering — ANN Gas Anomaly Model